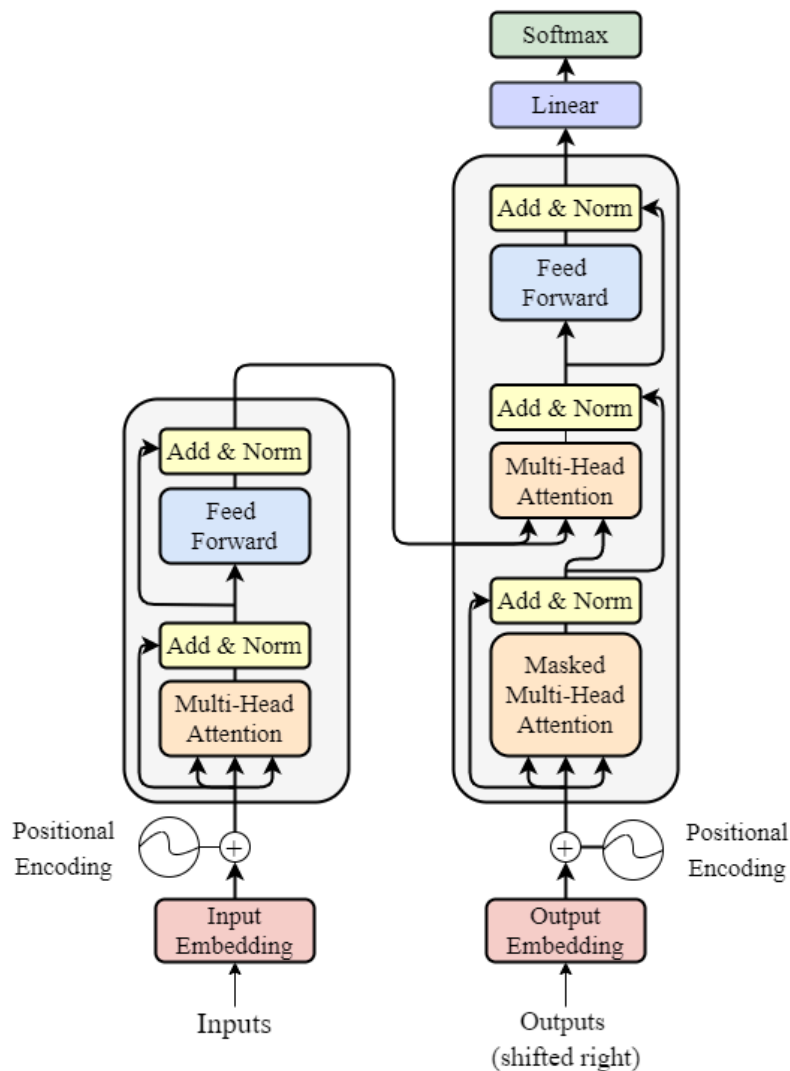




Introducción

En los últimos años no han pasado desapercibidas las inteligencias artificiales que de una manera similar a las películas más imposibles dentro de la ciencia ficción, son capaces de simular y generar textos unánimes a los de los humanos. Abarcando tanto la retórica como el humor y la prosa más metafórica característica de los seres racionales. No es de extrañar pues que ante esta aparición más o menos sopetada por el común de la sociedad, surgan debates y planteamientos filosóficos que cuestionan tanto su mecanismo interno como externo, desde cómo piensa hasta cómo creemos e interactuamos con algo que parece pensar. Sin embargo, más allá del debate metafísico que constantemente busca contornear los márgenes del

ánima, nos encontramos con que en el fondo existe también una ingeniería y artesanía igual o más fascinante e intrigante. Una arquitectura que si bien en su figura se ve sencilla, es capaz de sorprender hasta al **menos** ferreo informático o al más astuto matemático.

En este, cuaderno, trataremos de desderdiñar sus espinas y hacer un análisis general de los modelos TRANSFORMER donde se encuentra la semilla de las ias generativas de texto como CHAT-GPT, Copilot o el teclado de su dispositivo móvil. ¡Vamos a ello!

Arquitectura Transformer

La arquitectura transformer recibe este nombre, puesto que su misión consiste en transformar un vector de entrada, que es una representación vectorial de un archivo, a un vector de salida que es una mejor representación del archivo original. Si bien existen variaciones del modelo según su aplicación, hay dos bases fundamentales que más o menos todos siguen. La arquitectura se divide en dos fases, una encoder (codificador) y una decoder (decodificación). Esta arquitectura se diseñó por el año 2017 por Google para optimizar la traducción de textos. Para esta misión, la parte encoder sería la encargada de leer el texto, interpretarlo representando las palabras de la secuencia introducida. Este paso requiere de muchas fases para conseguir que la máquina, que lee 0s y 1s, sea capaz de leer palabras, comprender lenguaje e interiorizarlo para poder aprender de él y trabajar con él. La parte decoder en cambio sería la parte encargada de responder o crear la frase traducida. El encoder lee y comprende, el decoder responde. Sin embargo estas dos partes pueden tener diferentes pesos según la tarea queramos realizar.

En clasificación de textos es mucho mas importante entender el texto que no saber crear frases, pues la respuesta a la clasificación será una categoría (una palabra), por lo que nos fijaremos más en el encoder.

En las IAs generativas de texto es relevante entender el texto pero la parte más destacada es la de crear nuevo texto en base a lo previo. La primera palabra que generará la IA será a continuación de la pregunta, pero el resto de la oración se

generará a partir de la propia palabra generada y la suma de lo previo. Por eso gana más relevancia la parte del decoder.

Encoder:

Dado que estamos comunicándonos con máquinas no podemos esperar que lean tal cual como nosotros. Pues para ellas un signo de exclamación o un vocal representan simplemente un dígito sin ninguna diferencia expresiva. Es por ello que antes de enseñar al modelo a entender el texto debemos enseñarle a leer palabras y frases. Para ello tenemos que dotar al modelo de un lenguaje comprensible, a través de diferentes mecanismos. Lo principal es la división del texto en llamados tokens.

Nosotros los humanos estamos capacitados de discriminar contornos y espacios vacíos, lo que nos da la habilidad inata de distinguir unidades y conjuntos como uno. Es por ello que leemos palabra por palabra y no sílaba por sílaba. Pero en caso de las máquinas, como ya hemos mencionado, un carácter de espacio y un vocal o un signo no tiene una relevancia de distinguible más que su propio índice numérico. Es por ello que nosotros, los programadores, podemos decidir el conjunto de caracteres que definirá la unidad de lectura en la máquina. A esta unidad se le llama token. Y mientras que un texto "Hello. World" leído por un humano sería "[Hello][.][World]" una máquina podría leer "[Hel][llo][. W][orl][d]" o "[Hell][o. W][orld]" depende de la medida del token. Podríamos considerar modificar los tokens para que se asemejase al del humano y se centre en las palabras, sin embargo esto puede que en ocasiones no interese, pues una tokenización como la última puede brindarle a la máquina una cierta información de orden y funcionamiento de los caracteres de puntuación. Que la máquina aprenda, que tras una letra que acaba con el signo "." esta es seguida por un espacio y una letra mayúscula. Aunque también puede presentar desventajas si queremos optimizar la comprensión semántica de las palabras, ya que en tal caso los signos generarán ruido.

Una vez fragmentada la entrada hay que darles un valor, una coherencia y un orden a cada segmento. Un token de por sí solo no es más que un conjunto de unos y ceros.

Para este proceso existen dos procesos principales son denominados como "Embedding", ensamblar, y "Positional encoding", codificación posicional.

- **Embedding:**

A grandes rasgos el embedding sirve para agrupar y clasificar los tokens con algún rasgo similar entre sí, de modo que al procesar estos fragmentos no tenga que descifrar la definición de cada token uno a uno, si no que optimize el global del conjunto de similares. De este modo si el modelo lee la palabra "chalet" sabrá que probablemente el texto no está hablando de tipos de hormigas, si no tipos de hogar y reduzca su procesamiento de información a tokens que tengan una cierta **similitud semántica** como campo, descampado, casa, familia,... Para hacer un símil, sería muy poco eficiente que mientras alguien nos está hablando de su abuela, de cada palabra a otra que nos diga, nos tengamos que parar a pensar primero en todos los capitales de África, porque no tenemos un orden semántico estructurado.

Hay varios métodos de embedding dependiendo de si tratamos con imágenes u otros formatos, pero en textos se hacen uso de los word embedding. Entre los word embedding se pueden ver métodos como el Análisis de Componentes Principales (PCA), Embedding layer, Latent Semantic Analysis, que usó la descomposición de valores singulares, siendo de todos el más común el llamado Word2Vec. Todos ellos tienen como base relacionar los tokens semánticos con una representación vectorial.

- **Embedding layer:** El método genérico de word embedding es la aplicación básica de una capa de embedding donde se fijan dimensiones específicas donde serán representadas cada token. Esto es, a cada fragmento se le fija un vector de x dimensiones, por lo general significativamente inferior al número de palabras, y se le asignan valores aleatorios entre 0 y 1. De modo que mediante la optimización en base a la función de pérdida, los tokens más relacionados semánticamente estarán en un espacio de x dimensiones, más cerca entre sí. El embedding layer es flexible y se entrena conjuntamente con el resto del modelo.
- **Word2Vec:** Este es una técnica singular desarrollada por un equipo de Google en 2013. Como todos trata de convertir los tokens en vectores de característica numéricas de modo que palabras con significados similares también tengan una representación vectorial similar. Pero con la peculiaridad de que se valora más la verosimilitud semántica de las palabras enteras sin tener en cuenta los signos. Por lo que antes de realizar un Word2Vec es esencial excluir cualquier signo de los textos de entrada y dejar solo las palabras. El hecho de ser el más utilizado es que existen grandes redes preentrenadas con este método de embedding, ya que su eficiencia con grandes volúmenes de datos hace

que haya empresas que se dedican a preentrenar con grandes bases de datos. Sin embargo a su vez es más costosa que otras técnicas como el embedding layer clásico.

- **Positional encoding:**

Si bien es cierto que arquitecturas como las convolucionales o las recurrentes tienen en cuenta el orden o la posición de las entradas, la arquitectura Transformer no trae de por sí ningún método para guardarlo. Es por eso que se le añade una capa que defina la posición relativa o la absoluta de los tokens en la secuencia. A esta capa se le llama "positional encoding" y comparte las dimensiones con la capa de embedding de manera que pueden sumarse ambas. Como todo en las redes neuronales existen variantes, pero los más comunes son; la codificación posicional sinusoidal, la codificación posicional aprendida y la codificación posicional relativa. Siendo la más habitual la sinusoidal.

- **Sinusoidal Positional Encoding:** Este es el método utilizado en el artículo original "Attention is All You Need". Se encarga de asignar a cada posición en los tokens de entrada mediante una frecuencia sinusoidal específica para cada una. Primero usando la función del seno para dimensiones pares y el coseno para las impares. Una vez se calcula, esta matriz de vectores de codificación posicional se suman a los embeddings de los tokens. De este modo, ya no solo tienen una representación semántica, sino también una representación de su posición en la secuencia. La fórmula para una dimensión $2i$ (par) y una $2i+1$ (impar) es:
 - P
 - E
 - $\text{pos}, 2i$
 - $= \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$
 - pos
 - 10000
 - $2i$
 - d
 - model
 - $)$
 - P
 - E
 - $\text{pos}, 2i+1$
 - $= \cos\left(\frac{\text{pos}}{10000^{\frac{2i+1}{d}}}\right)$
 - pos
 - 10000
 - $2i$

- d
- $model$

○ $)$

- Donde:
 - pos es la posición.
 - i es la dimensión del vector de codificación.
 - $d_{\{model\}}$ es la dimensión total del vector de codificación, o, la dimensión del modelo de embeddings.
- Este metodo tiene una ventaja clave que cabe mencionar. Como se puede observar, no requiere de ningún entrenamiento, ya que, las codificaciones son fijas y no necesitan ser ajustadas. Por lo que es simple y eficiente.

Una vez transcrito el texto de entrada al lenguaje de las máquinas es donde empieza la base de la red neuronal. Que si bien el conjunto de la arquitectura es característica de un transformer, es la siguiente combinación de técnicas la que transforma el texto de entrada en algo de valor. Es la parte en la que la máquina aprende a entender los textos capturando las dependencias y relaciones complejas entre sus elementos. Este apartado gana el nombre del modelo llamandose transformer layer y se compone principalmente de mecanismos de atención y de (feed forward)

- **Attention:**
 La atención, Multiple Head Attention, es el corazón del transformador. Es la encargada de dar a cada token un valor dentro del texto. Tomando todas las entradas preprocesadas y contextualizando cada una de estas dependiendo de los tokens que le rodean. Dependiendo de la función que se quiera cumplir, la atención puede estar enfocada en el global del texto, en los k -tokens más próximos o incluso en combinaciones de tokens cumulativos. Sin embargo, dentro de los mecanismos de atención el "self-attention dot product" es el más sonado. Este en su forma más sencilla busca el valor de cada token en referente a la combinación del resto de tokens en el texto. La atención es un tema complejo de interpretar, pero intentaremos desarrollarlo de la manera más sencilla basandonos en el artículo original "Attention is all you need".
 Mencionar que el Multi-Head Attention consiste en varias capas de atención procesandose en paralelo.
 Una vez tenemos los vectores de embedding que representan las posiciones de cada token en un espacio de x dimensiones la capa de atención trata de relocalizar estos tokens condicionandolos por las posiciones de los tokens que les preceden, para generar una posición que represente mejor la combinación de las características.

De manera más ilustrativa, un token, digamos, "Torre" se encontrará cerca de palabras como "alto", "edificio" o "castillo" en el espacio de los embedding. Una localización algo general y abstracta. Sin embargo, si el token previo fuera "París", un token que probablemente se encuentra lejos de "Torre", rodeado de capitales, la capa de atención triangularía una nueva coordenada determinada para la Torre Eiffel. Mientras que si el token previo fuera "Alfil", desplazaría las coordenadas de "Torre", a unas nuevas coordenadas lejos de "alto", "edificio" o "Eiffel", y recolocaría en un nuevo conjunto de coordenadas caracterizada por el ajedrez.

Para conseguir esta transformación del espacio vectorial de embeddings el autor del artículo hace uso de tres matrices que estiman mediante capas densas o *dense layers*, fully connected; Q (query), K (key) y V (values). V, values, son los vectores embedding a los que se les aplica una dense layer para proyectar en un espacio más específico y mejorar la representatividad de los tokens individuales y facilitar la combinación con el resto de matrices. Mientras que la query y la key son las encargadas de dar el marco narrativo al token. Se pueden entender las queries como vectores que contienen las probabilidades de que cada token haya sido precedido por el resto de tokens y la key como el conjunto de vectores que guardan las probabilidades de que sean seguidas por los siguientes tokens. Salvando las distancias de que en realidad las matrices Q y K no guardan probabilidades como tal, si no nuevos espacios que representan los conjuntos de tokens más afines. Para luego calcular su producto vectorial o "dot product" por el que recibe el nombre, donde los tokens más probables a ser consecutivos tendrán valores mucho más elevados que el resto, consiguiendo que la máquina centre su atención. Pueden llegar a ser tan elevados que conviene dividir por la raíz cuadrada de la dimension de K para estabilizar los gradientes. Pues, tras escalar, se aplica una softmax que generará una matriz estocástica, ahora sí, de probabilidades (sumaran 1).

Finalmente, las puntuaciones de atención se utilizan para ponderar los valores de V, produciendo la salida final de la atención. Una representación espacial más abstracta del texto y más representativa semánticamente.

- Attention(Q,K,V)=softmax(
 - Q
 - K
 - T
 - d
 - k
 - $\sqrt{}$
 -)V
-

- **Feed-Forward Networks:**

Tras pasar por el bloque de atención, la red neuronal consigue contextualizar los tokens, o palabras, entre sí de una manera bastante acertada. Sin embargo antes de salir del modelo, pasa por una capa "feed-forward" que sirve como un filtro de drenaje, ayudando a crear patrones más complejos entre los datos y rutas más eficientes de procesamiento. "Es distinto saber leer un mapa o saberse el camino optimo en el mapa con solo abrirlo".

En la arquitectura transformer, este bloque se compone de operaciones lineales y no-lineales encadenadas una tras de sí, aplicando una *dense layer* y una función de activación consecutivamente a los datos transformados.

Optimizando ligeramente los pesos y los biases de las operaciones lineales para conseguir un mejor modelo.

En el artículo original representado como:

- $FFN(x) =$

- W

- 2

- $\cdot \text{ReLU}(\cdot)$

- W

- 1

- $x +$

- b

- 1

- $) +$

- b

- 2

- x

- es el vector de entrada.

- W

- 1

- y

- W

- 2

- son las matrices de pesos.

- b

- 1

- y

- b

- 2

- son los vectores de bias.

- ReLU es la función de activación no lineal.

Decoder:

El decoder es otra estructura transformers que contiene las mismas partes que contiene el encoder, pero con un añadido: una capa "masked-self attention", que es una variante de la capa self attention, con el objetivo de tener en cuenta la causalidad de las palabras. Es decir, para que los tokens posteriores en el texto no influyan en la imagen de las primeras frases, que la lectura sea de izquierda a derecha. Esto se logra fijando a los valores inferiores de la diagonal

Q

K

t

un valor negativo extremo, "infinito", de modo que al aplicar la softmax queden en cero. Siendo la diagonal inferior de la matriz, la aportación de los tokens futuros a los previos.

Esta configuración no es necesaria en el encoder, puesto que es bidireccional y lee las frases en ambas direcciones. Sin embargo, el decoder se encarga de generar la secuencia output (frase), y las frases se generan hacia delante. Para que generemos frases con sentido, es necesario que las palabras se vayan generando teniendo en cuenta las que generará después, y para que el decoder sea capaz de hacerlo, es necesaria esta capa "masked-self attention".

Output:

Una vez se pasan todas las transformaciones, se aplica una dense layer y mediante una función de activación que dependerá del objetivo, se obtienen las probabilidades de salida. En el caso de un generador de textos, la palabra próxima más probable mediante una softmax o si es un clasificador de dos posibles niveles una sigmoide. Que indicará la probabilidad de pertenecer a una cierta clase u otra.

Refinamiento:

- **Residual Connection y Normalization:**

En el marco teórico, las Redes Neuronales mejoran su actuación al añadir más capas, pero en la práctica, la mejora empieza a decrecer tras un cierto punto. Esto se debe parcialmente a que el signo del gradiente fluyendo en el Backward (técnica de refinado de parámetros mediante el descenso del gradiente) puede llegar a planicies donde se convierten inestables cuando el número de parámetros es muy grande. Es por eso que la conexión residual se utiliza para mejorar el flujo del gradiente, permitiendo fluir por un camino más directo del output a la primera capa de la red en el backward. Para conseguir esto, en cada capa que transforme los datos, se conecta parte de los datos originales con la salida de la función. De esta forma, el gradiente logra una conexión más directa con los parámetros de las primeras capas.

Por otro lado, la Normalización de las capas también ayuda a acelerar y estabilizar el entrenamiento. Además ayuda a que las representaciones internas de la red sean consistentes a lo largo del tiempo, lo que es crucial teniendo en cuenta la cantidad abrumadora de vectores que surgen tras realizar la operación de atención. La normalización se aplica tras añadir las conexiones residuales en todas las subcapas.

Output: Una vez se pasan todas las transformaciones, se aplica una dense layer y mediante una función de activación que dependerá del objetivo, se obtienen las probabilidades de salida:

Como hemos dicho arriba:

- **BERT:** En la clasificación de textos es importante entender, por lo que nos fijamos en el encoder. Para este tipo de problemas se presentó la arquitectura Bidirectional Encoder Representations from Transformers (BERT). Esta arquitectura contiene solo el encoder de la arquitectura original. Añade unos tokens especiales (cls) al inicio de cada secuencia input (frase/documento), y se entrena para predecir palabras enmascaradas de la secuencia input, como podría ser el sentimiento de un texto p.e. Después de el pre-entrenamiento podemos mejorar el modelo para que prediga la palabra enmascarada de la secuencia entera, usando la codificación del token especial (cls), o para que prediga cada palabra de la secuencia, usando las codificaciones de cada palabra.
- **GPT:** En las IAs generativas de texto es importante generar, por lo que nos fijamos en el decoder. Para este tipo de problemas se presentó la arquitectura Generative Pre-trained Transformer (GPT). Esta arquitectura contiene solo el decoder de la arquitectura original. Para hacerlo, utiliza la masked

self-attention y elimina la cross attention del bloque decoder. Se entrena para predecir la siguiente palabra de una secuencia input.

El output de la red serán unas probabilidades asociadas a una palabra. En el caso de un problema de clasificación usando BERT, es decir solo con el encoder, estas probabilidades se asociarán a las categorías que queremos predecir, y se aplicará la función de activación Softmax. Gracias a esta función las salidas sumarán uno, y podremos hablar de probabilidades. El output de esta red serán las diferentes categorías con la probabilidad de que dicho input sea de cada una de estas categorías. Tendremos que elegir que categoría predice la red, y esto lo podemos hacer con una única categoría (la que tenga más probabilidad), o con las tres categorías más probables, o como nosotros queramos.

En el caso de un problema de generación de texto usando GPT, es decir solo con el decoder, estas probabilidades se asociarán a las posibles palabras que vienen a continuación de la palabra actual. El output de la red será una de esas palabras, y a partir de esta volverá a calcular las probabilidades de la siguiente, y así ira generando palabra a palabra (o token a token) la frase output. El Generador puede seleccionar la siguiente palabra siguiendo diferentes criterios:

Generar la palabra más probable (greed sampling), generando la secuencia más probable (Beam Search), o generar al azar la siguiente palabra (sampling), eligiendo entre las k más probables (top-k sampling) o entre las k palabras que acumulan el p% de probabilidad (top-p sampling). Gracias al sampling las IAs generativas responden de diferentes formas a una mismo input.

En Bert, solo habrá un input (el texto) y un output (la clase predicha), mientras que en el GPT, tendremos un input (la frase que respondera el generador) y varios outputs (la siguiente palabra que genera la red) que sirven de input para el siguiente paso de la generación.

[]

Demostración

Para poner un ejemplo práctico de todo lo anterior, realizaremos un modelo de la arquitectura BERT y lo entrenaremos para que sea capaz de clasificar comentarios de usuarios de IMDB sobre películas. Introduciremos un texto con el comentario de algun usuario sobre una película, y el modelo tendrá que classificar ese texto según si al usuario le ha gustado la película (su comentario es positivo) o si le ha disgustado (su comentario es negativo).

Para entrenar esta red utilizaremos un conjunto de datos descargado desde la página de kaggle, que contiene 50000 comentarios de películas, y sus 50000 etiquetas correspondientes, que tienen dos posibles valores: positivo / negativo